AMDox ERP

Enterprise Resource Planning System — Project Report

Author	Om Jyoti Prakash
Date	31 October 2026
Live application	https://amdox-erp-psi.vercel.app
Source code	https://github.com/omjyotiprakash/amdox-erp
Tech stack	React, Node.js, Express, MongoDB (MERN)

Status: Complete and live

1. Overview

AMDox ERP is a full-stack web application that helps a company run its daily work in one place: finance, human resources, stock and suppliers, projects, reports and administration. It works on computers and on mobile phones, and it is live on the internet.

The problem

Small and medium businesses often keep their accounts, staff records, stock and projects in separate spreadsheets and tools. Information is scattered, totals are worked out by hand, and anyone with the file can see or change everything.

The solution

- One application with separate modules for each area of the business.
- Every screen works with real data saved in a database.
- Each person gets a role that decides exactly what they can see and change.
- Live summaries and alerts, so problems (overdue invoices, low stock, late projects) are noticed early.

Objectives and result

Objective	Result
Central place for finance, HR, supply chain and projects	Done — 6 modules, 22 screens
Real, saved data instead of sample data	Done — MongoDB database
Secure login and role-based permissions	Done — checked on the server
Works on desktop and mobile	Done — app-style mobile layout
Deployed online for others to use	Done — Vercel, Render, Atlas
Reliable and maintainable	Done — 120 automated tests run on every change

2. Technology Stack

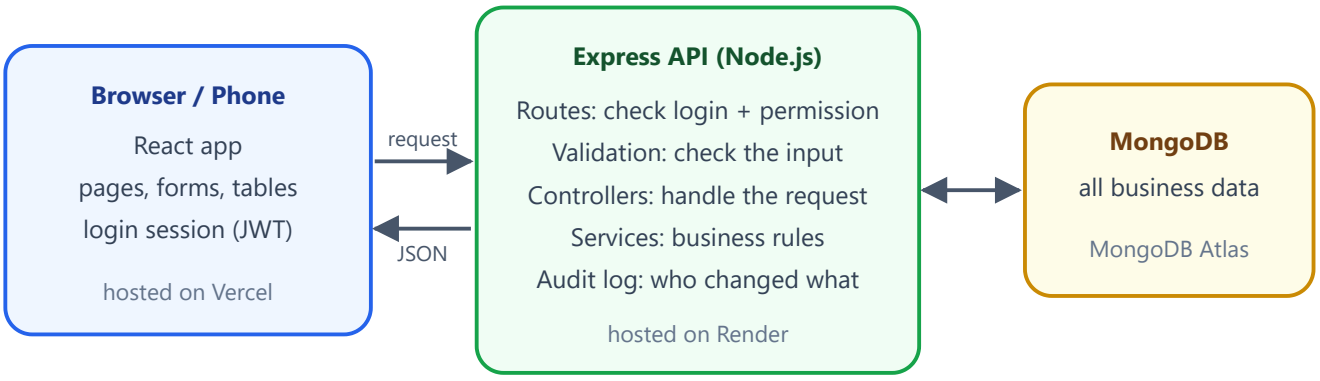
Layer	Technology	Used for
Frontend	React 19, Vite	The user interface and fast production builds
Styling	Tailwind CSS	Responsive layout for desktop and mobile
Routing	React Router	Moving between pages, protected pages
Backend	Node.js, Express 5	The REST API and business rules
Database	MongoDB Atlas, Mongoose	Storing all business data
Security	JWT, bcrypt, Helmet, rate limiting	Login, password protection, safe headers
Validation	Zod	Checking every input on the server
Testing	Node test runner, Vitest, Testing Library	Automated backend and frontend tests
Hosting	Vercel, Render, MongoDB Atlas	Website, server and database in the cloud
Automation	GitHub Actions	Runs all tests and the build on every pull request

Development approach

The work was split into small parts, each built on its own Git branch and merged through a pull request: authentication, finance, HR, supply chain and projects, dashboard and settings, security hardening, access control, and the mobile layout. Each part was tested before it was merged.

3. Architecture

The application has three parts that talk to each other over the internet.



All traffic uses HTTPS. Permissions are enforced on the server, not only hidden on the screen.

How a request works

- The user opens a page; React shows it and asks the API for data.
- The API checks the login token and the person's role permissions for that module and action.
- The input is validated (dates, amounts, required fields) before anything is saved.
- The service applies the business rules, reads or writes MongoDB, and returns JSON.
- Successful changes are written to the audit log (who, what, when — never the data itself).

Code structure

Folder	Contents
backend/src/routes	URLs, input validation and permission checks
backend/src/controllers	HTTP request and response handling
backend/src/services	Business rules (totals, approvals, payroll, forecasting...)
backend/src/models	Database models for every record type
frontend/src/pages	One screen per page, grouped by module
frontend/src/components	Shared parts: layout, popups, tables, pagination
frontend/src/services	All calls to the API in one place

4. Modules and Features

Module	Features
Login and users	Secure login, closed public sign-up (Admins create accounts), change password, roles with View / Create / Edit / Delete permissions per module, custom roles.
Dashboard	Live revenue, staff, project and approval figures; a "needs attention" list; notification bell (overdue invoices, low stock, pending approvals, late or over-budget projects).
Finance	General ledger with income/expense totals; accounts payable and receivable with overdue detection and "mark as paid"; monthly reports with profit margin and CSV export.
Human Resources	Employee directory; daily attendance with check-in/out; leave requests with approval (nobody can approve their own); monthly payroll generated from salaries (Pending → Processing → Paid).
Supply Chain	Inventory with automatic stock status; vendors; purchase orders with a forward-only status flow; demand forecast with reorder suggestions.
Projects	Projects with budget, progress and milestones; resource planning that never books a person above 100%; project team shown on the project page.
My Profile	Every employee sees only their own details, attendance and payslips, and can request or cancel their own leave.
Settings	Company settings, roles and permissions, user accounts, integrations list, audit log.

Important business rules

- Totals count only posted transactions; "overdue" is worked out from the due date, so it is never out of date.
- Records with history (an employee with payroll, a vendor with orders) cannot be deleted — they are set to inactive instead.
- Payroll copies the salary at the time it is generated, so later salary changes do not rewrite history.
- The last active Admin can never be removed, so the system cannot be locked out.

5. Security, Mobile and Quality

Security

- Passwords are hashed with bcrypt; login sessions use signed tokens (JWT).
- Every request is checked on the server against the person's role permissions.
- Public sign-up closes after the first Admin account; Admins create all other accounts.
- Changing or resetting a password signs that person out on all other devices.
- Login attempts are rate limited; every input is validated; secure HTTP headers are set.
- Staff can only ever see their own personal records in My Profile.
- An audit log records who created, changed or deleted records, and when.

Mobile experience

- App-style layout: bottom tab bar, compact header with an account menu.
- Tables turn into easy-to-tap cards; every popup slides up from the bottom of the screen.
- It can be installed on a phone's home screen like a normal app.

Testing and quality

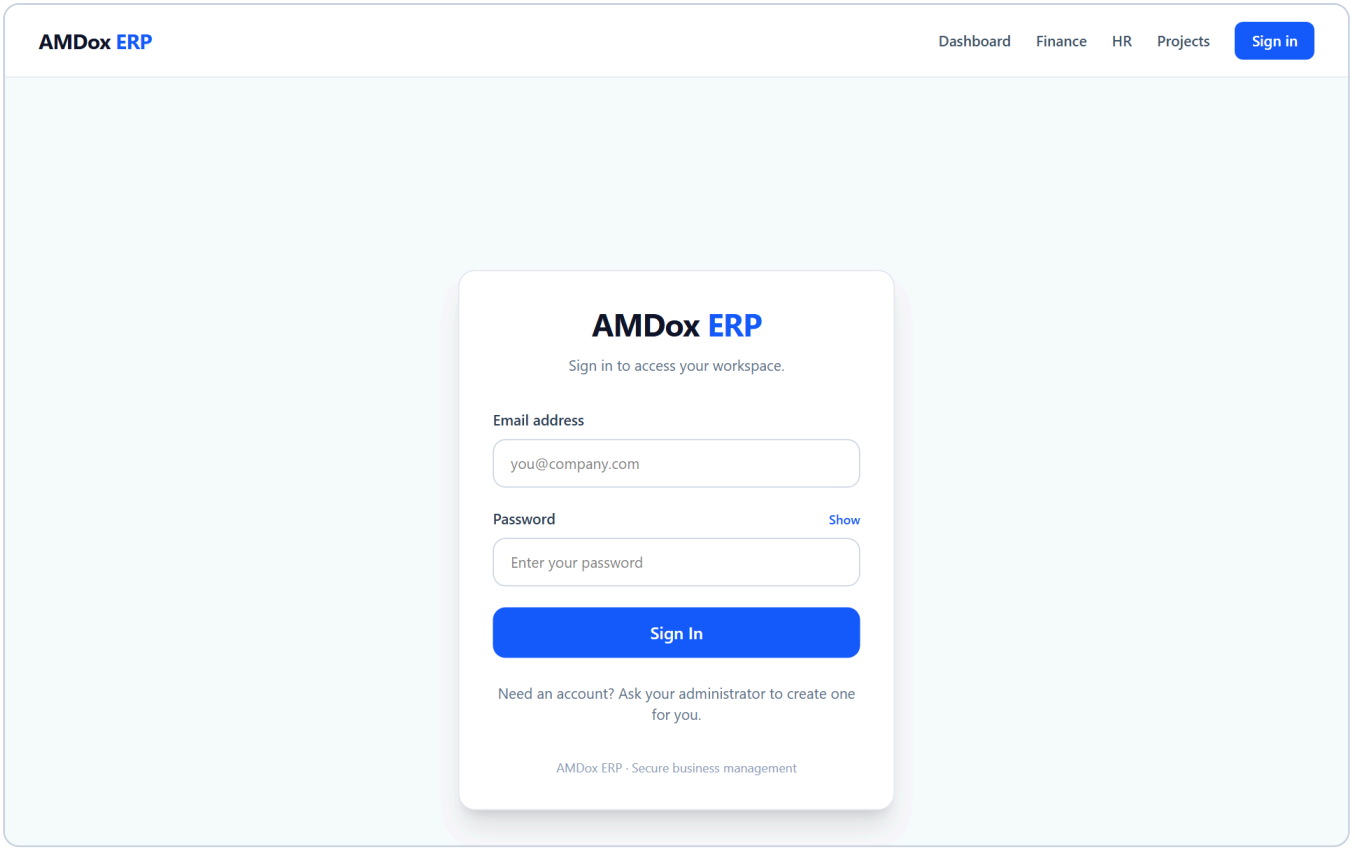
Area	Details
Backend tests	63 automated tests: permissions, money totals, approvals, payroll, privacy, pagination, sessions, backups
Frontend tests	57 automated tests: forms, popups, navigation, permissions in the menu, API client
Continuous integration	GitHub Actions runs all tests and the production build on every pull request
Manual check	All main screens checked in desktop and phone-sized browsers against the real backend
Backups	Backup and restore scripts for the database

Deployment

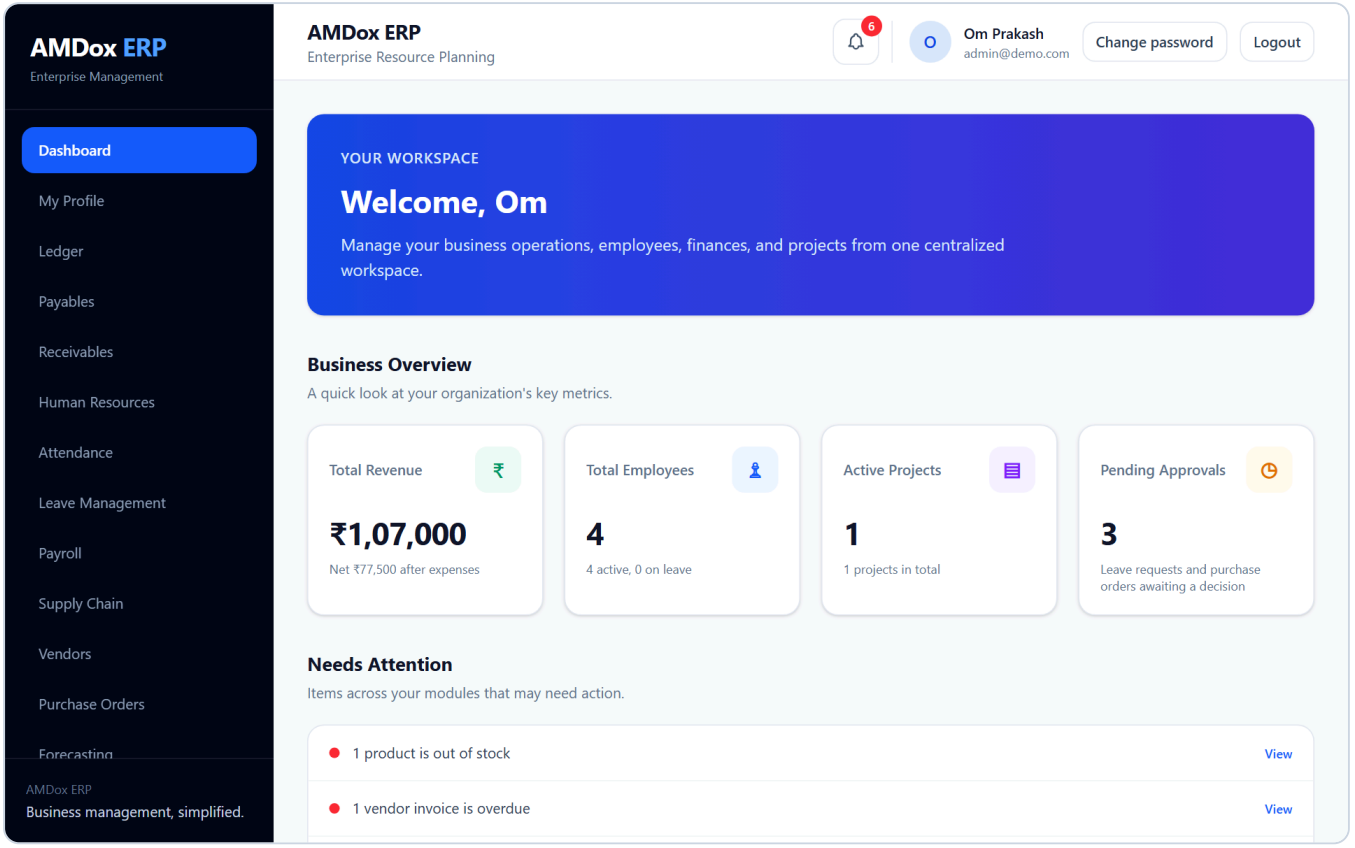
Part	Service	Address
Website (frontend)	Vercel	https://amdox-erp-psi.vercel.app
API (backend)	Render	https://amdox-erp-api-st5o.onrender.com/api/health
Database	MongoDB Atlas	Cloud database (private)

When code is merged on GitHub, Vercel and Render rebuild and publish it automatically.

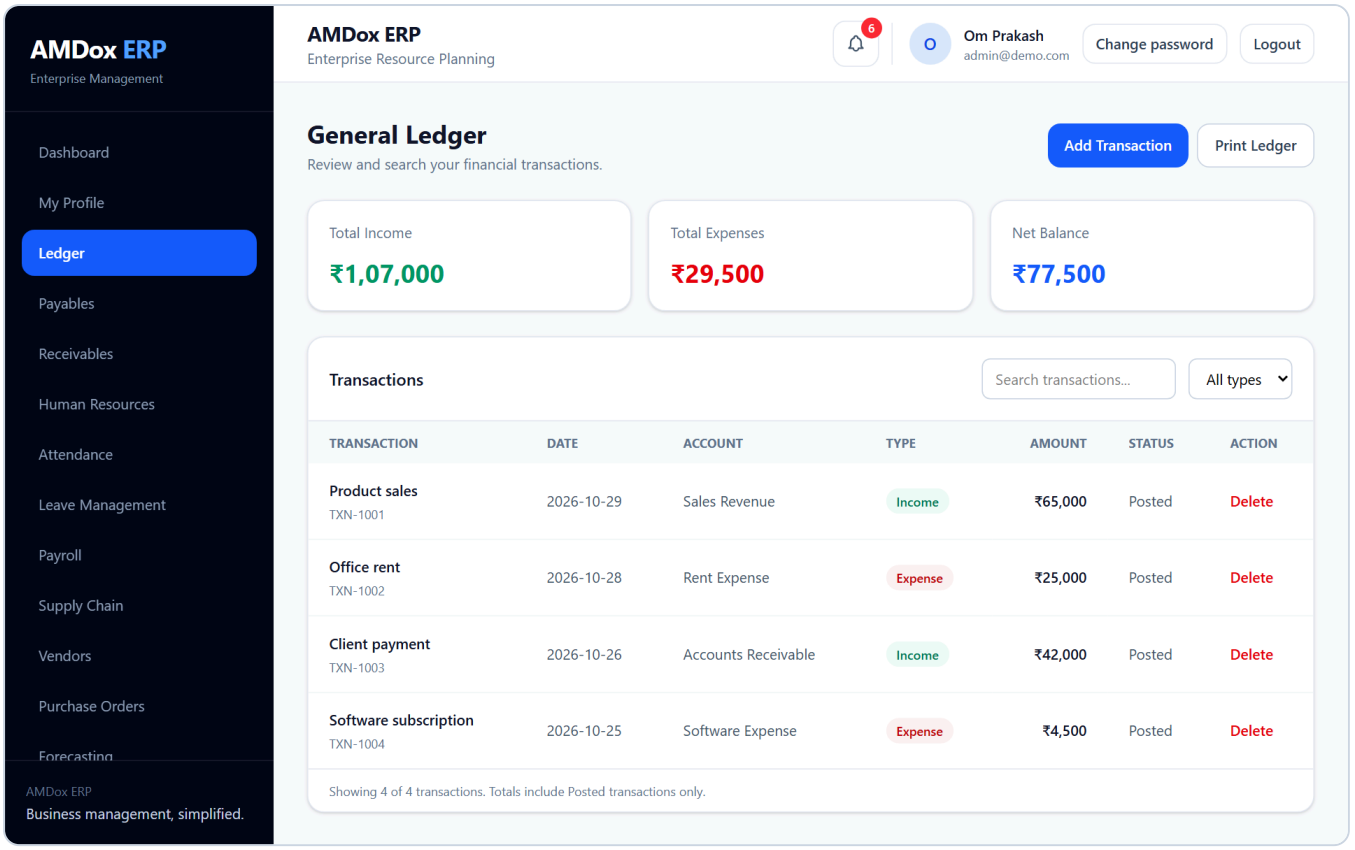
6. Screenshots



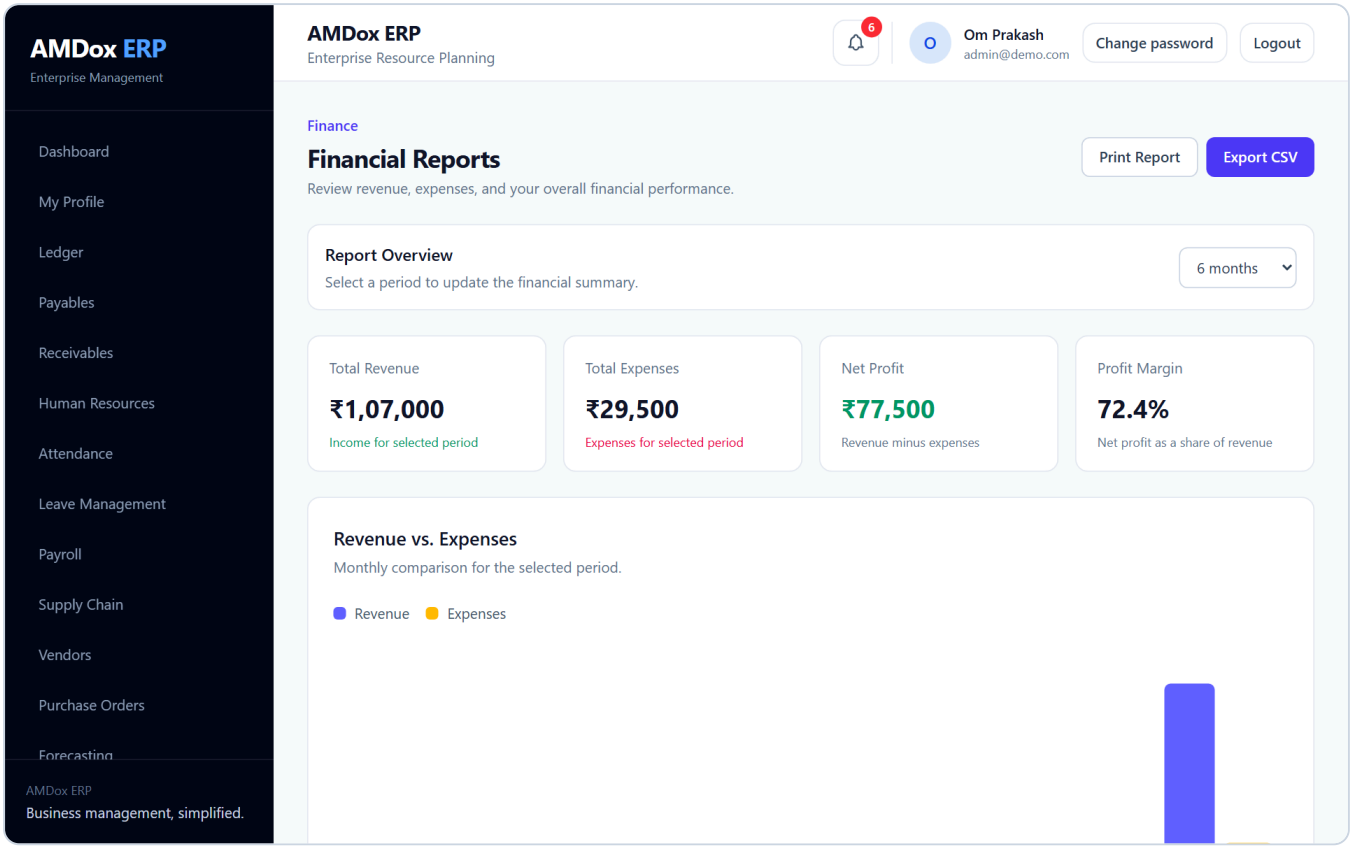
Login page



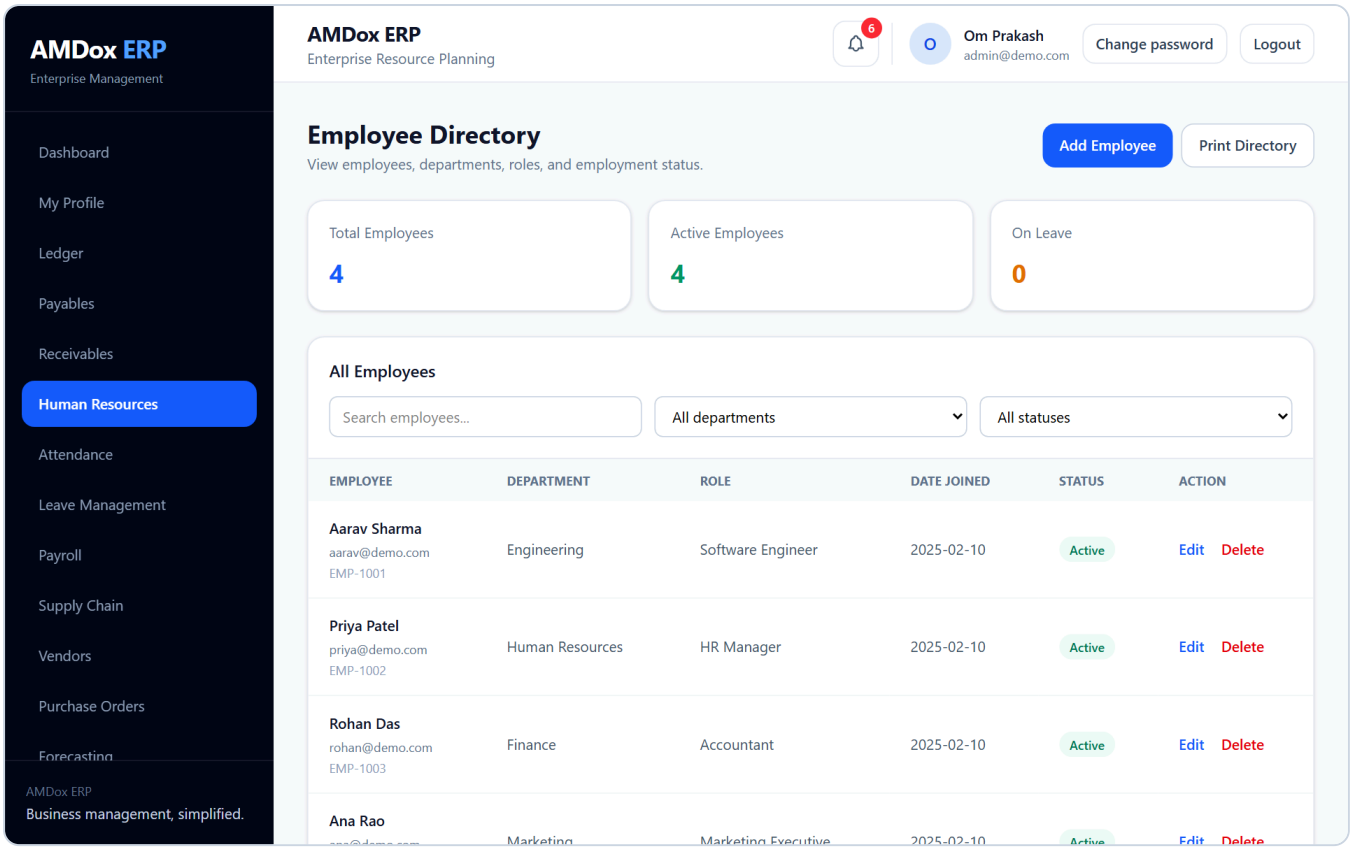
Dashboard with live figures and the "needs attention" list



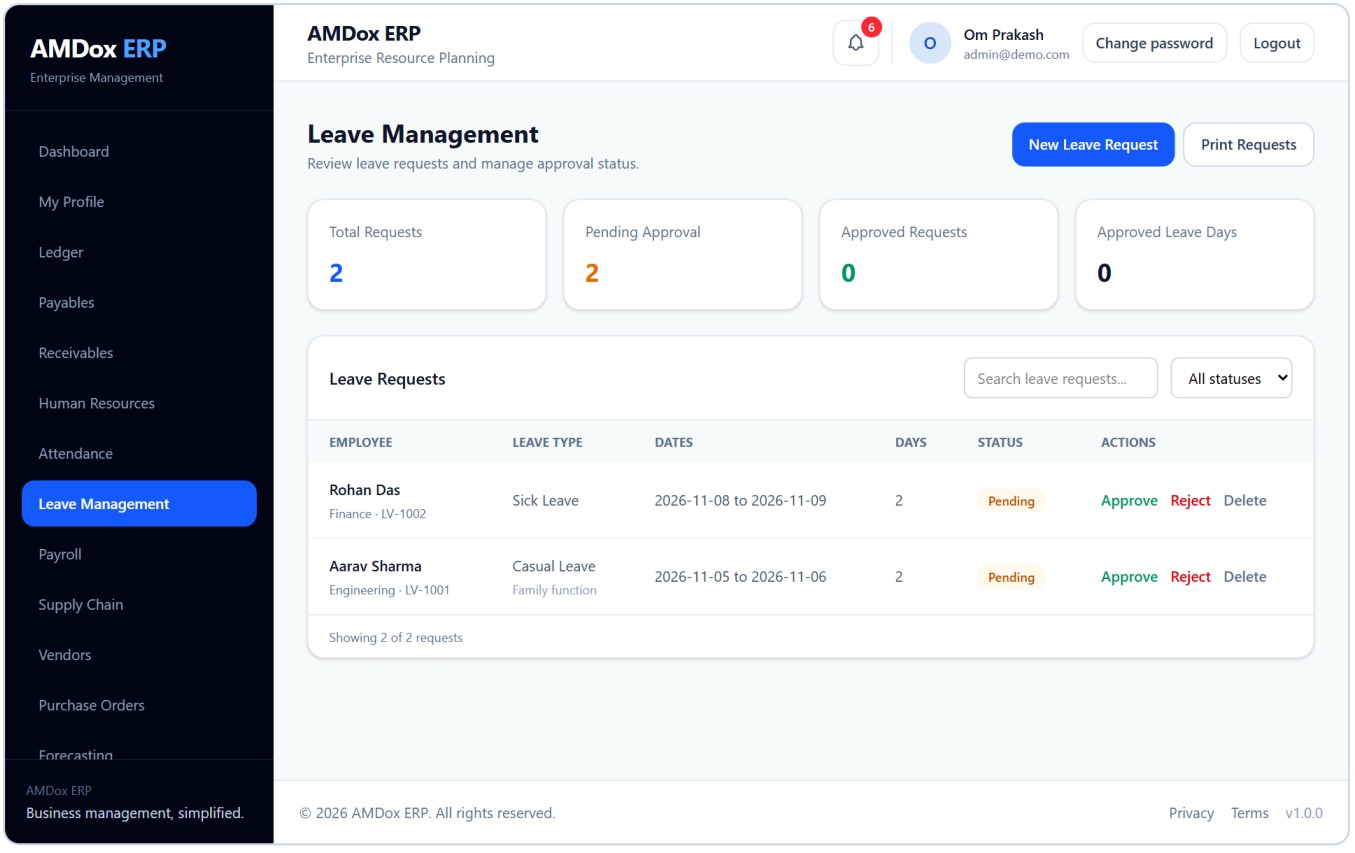
Finance — General Ledger



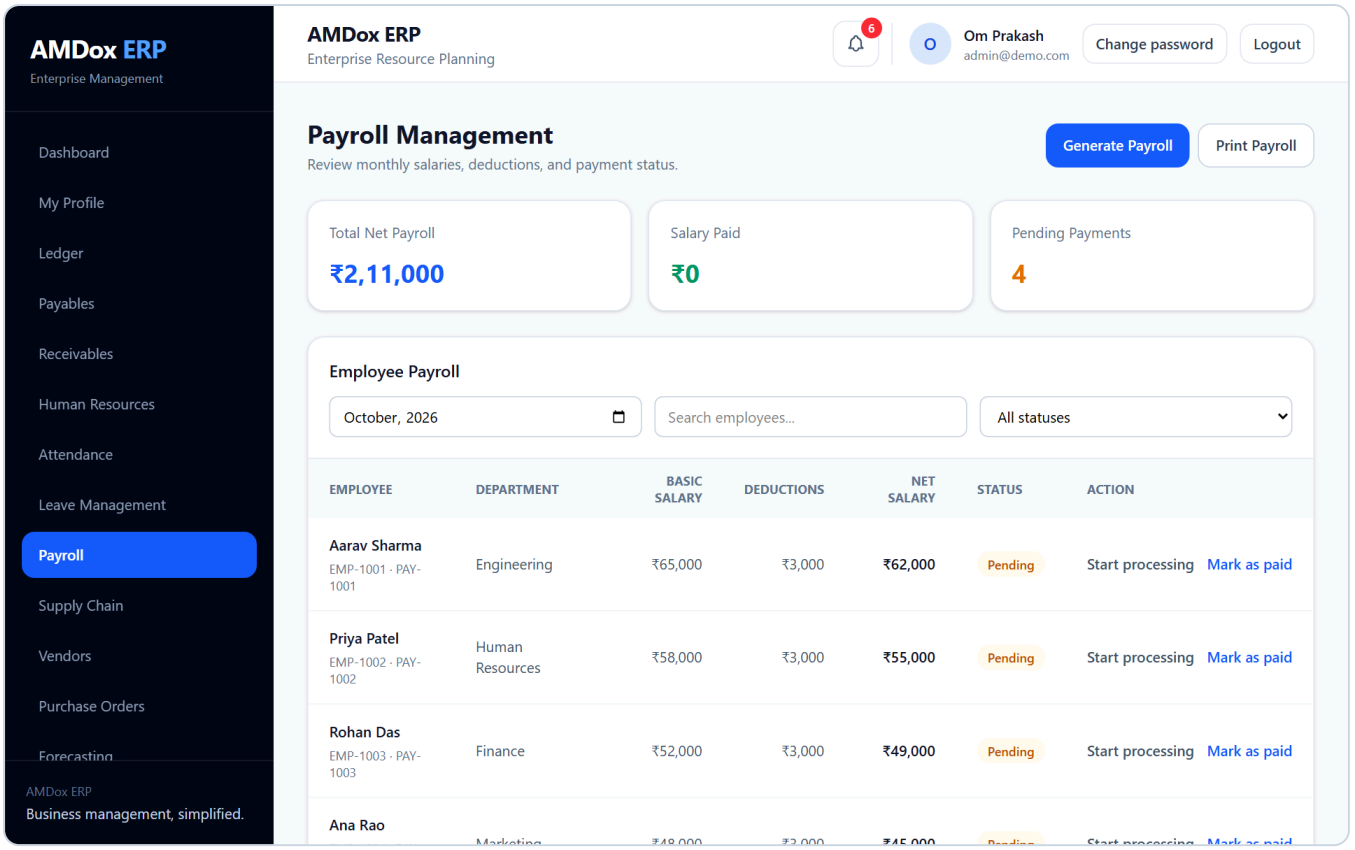
Finance — Financial Reports



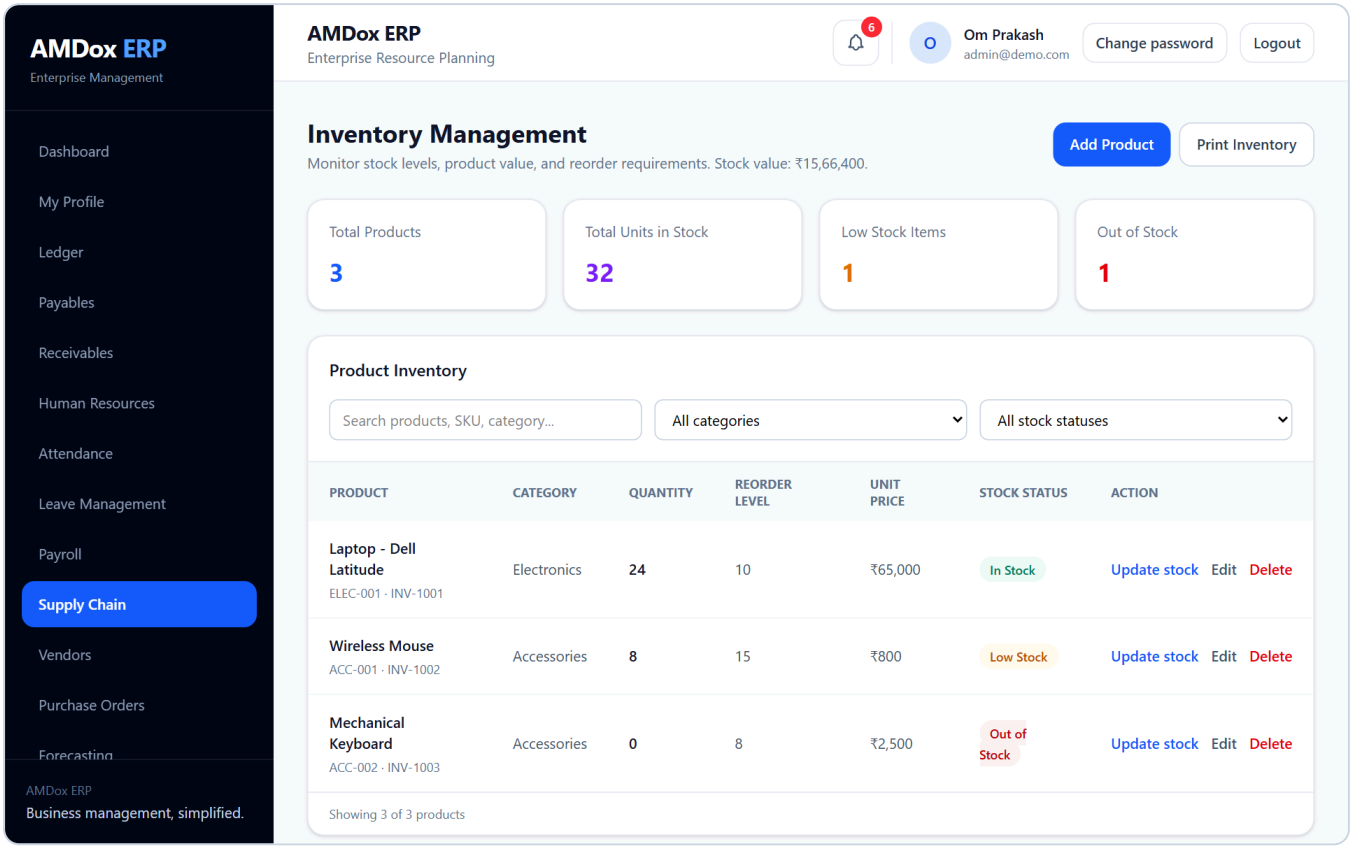
HR — Employee Directory



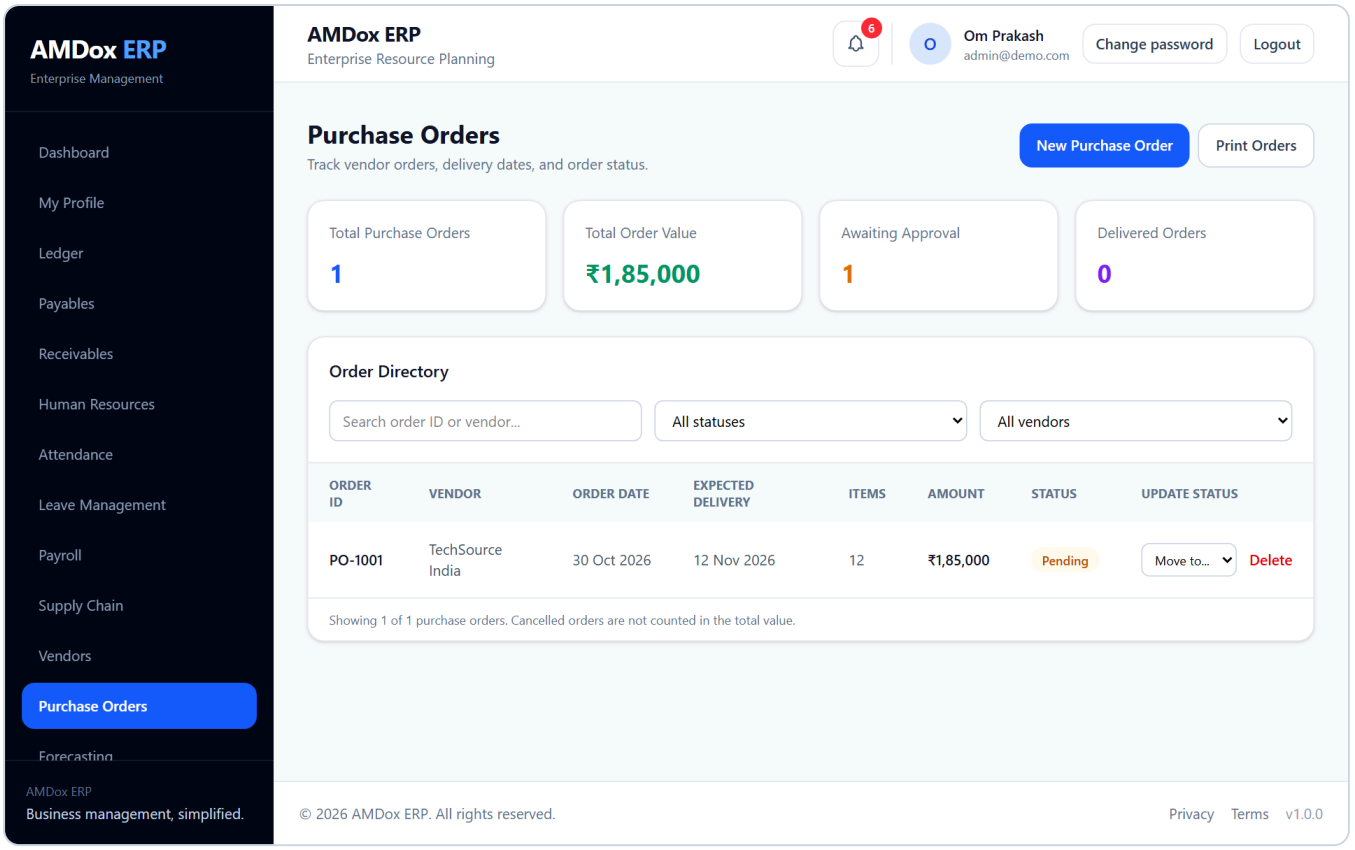
HR — Leave Management with approvals



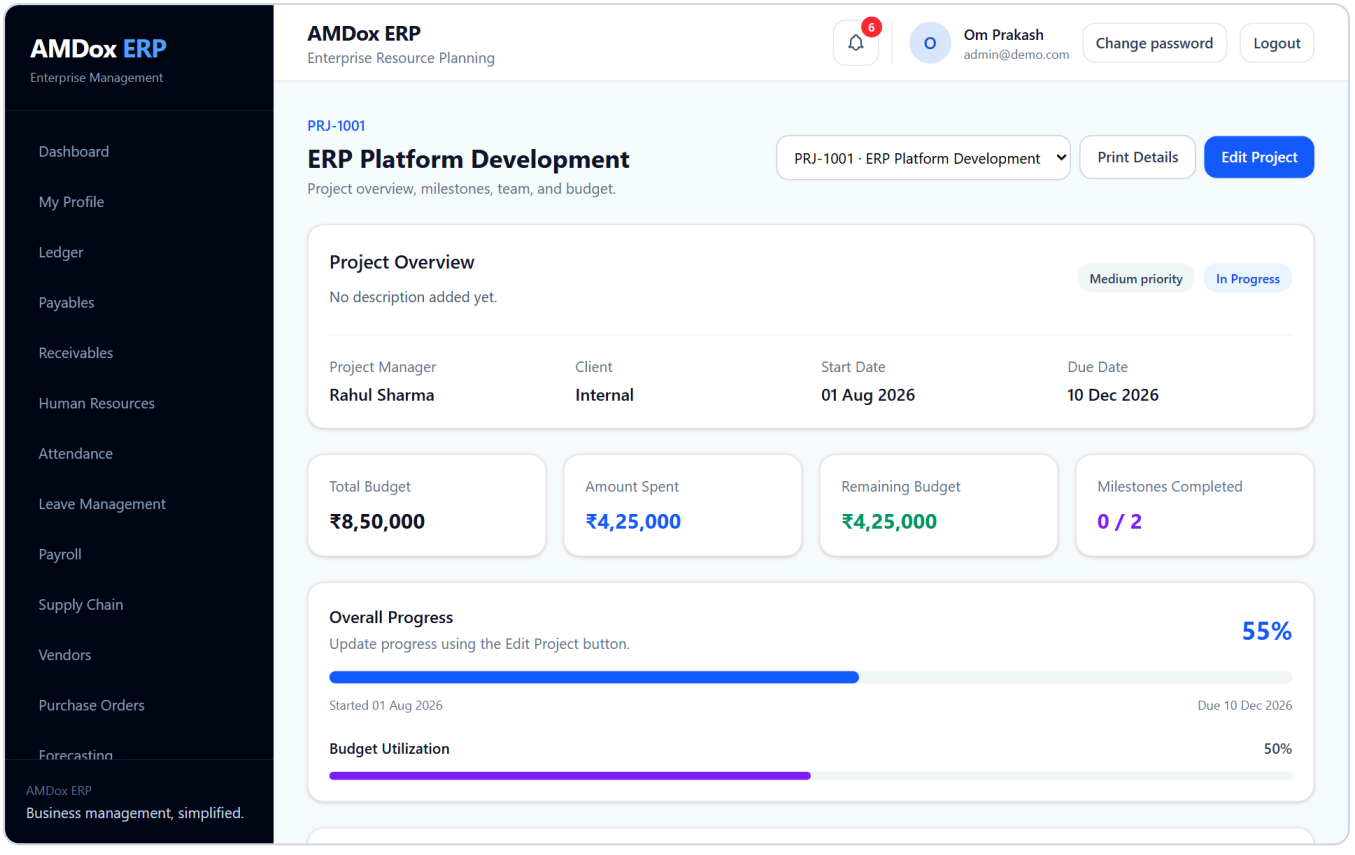
HR — Payroll



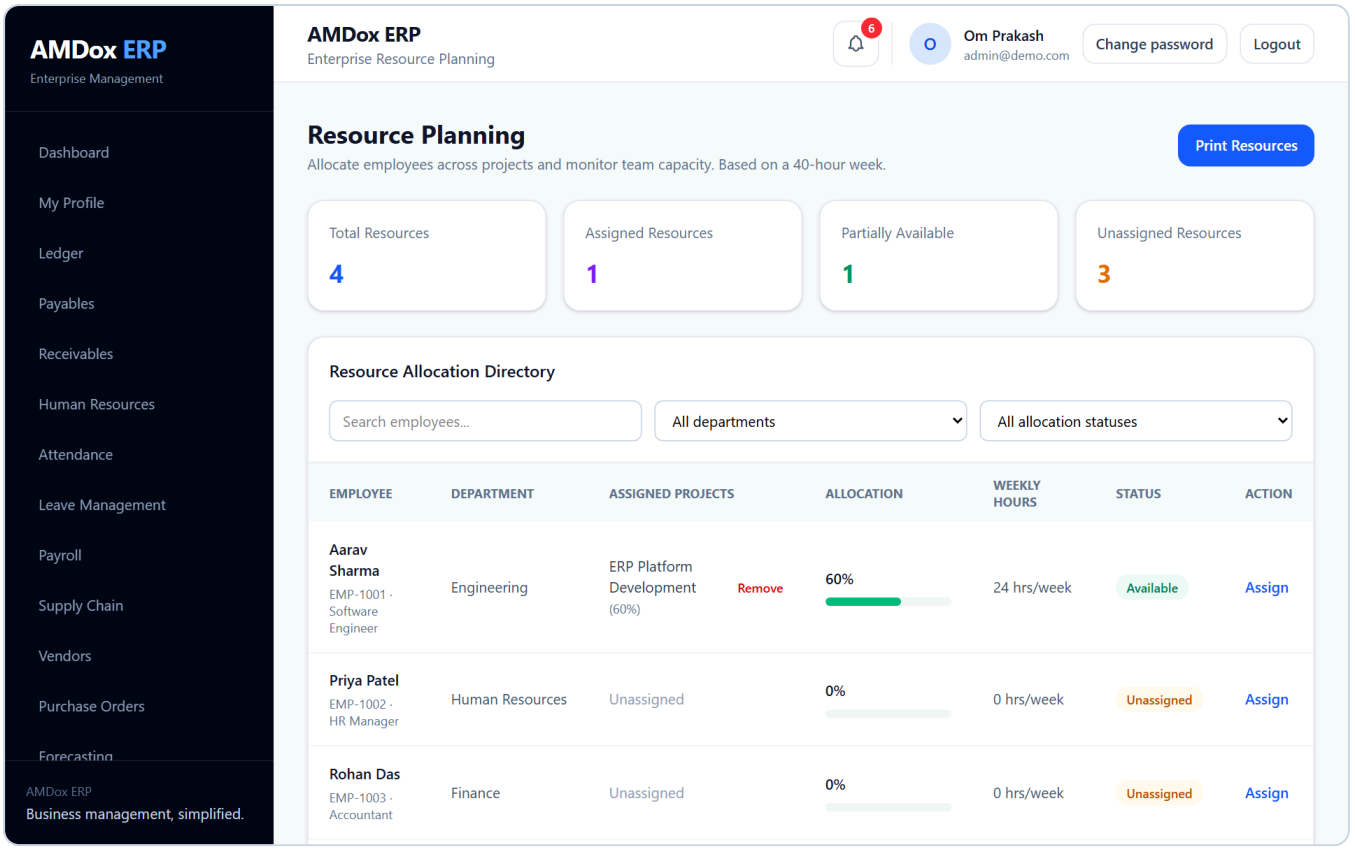
Supply Chain — Inventory with stock status



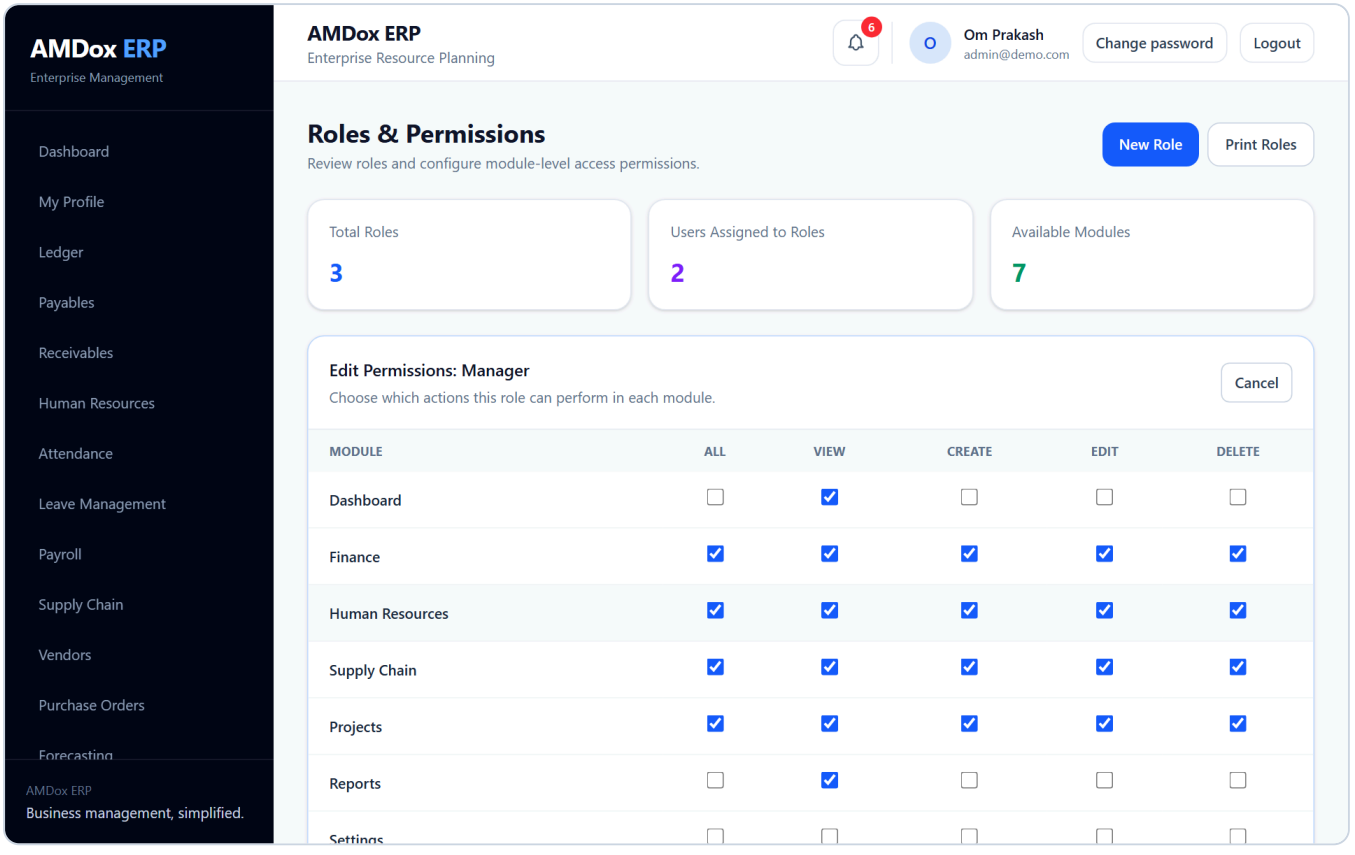
Supply Chain — Purchase Orders



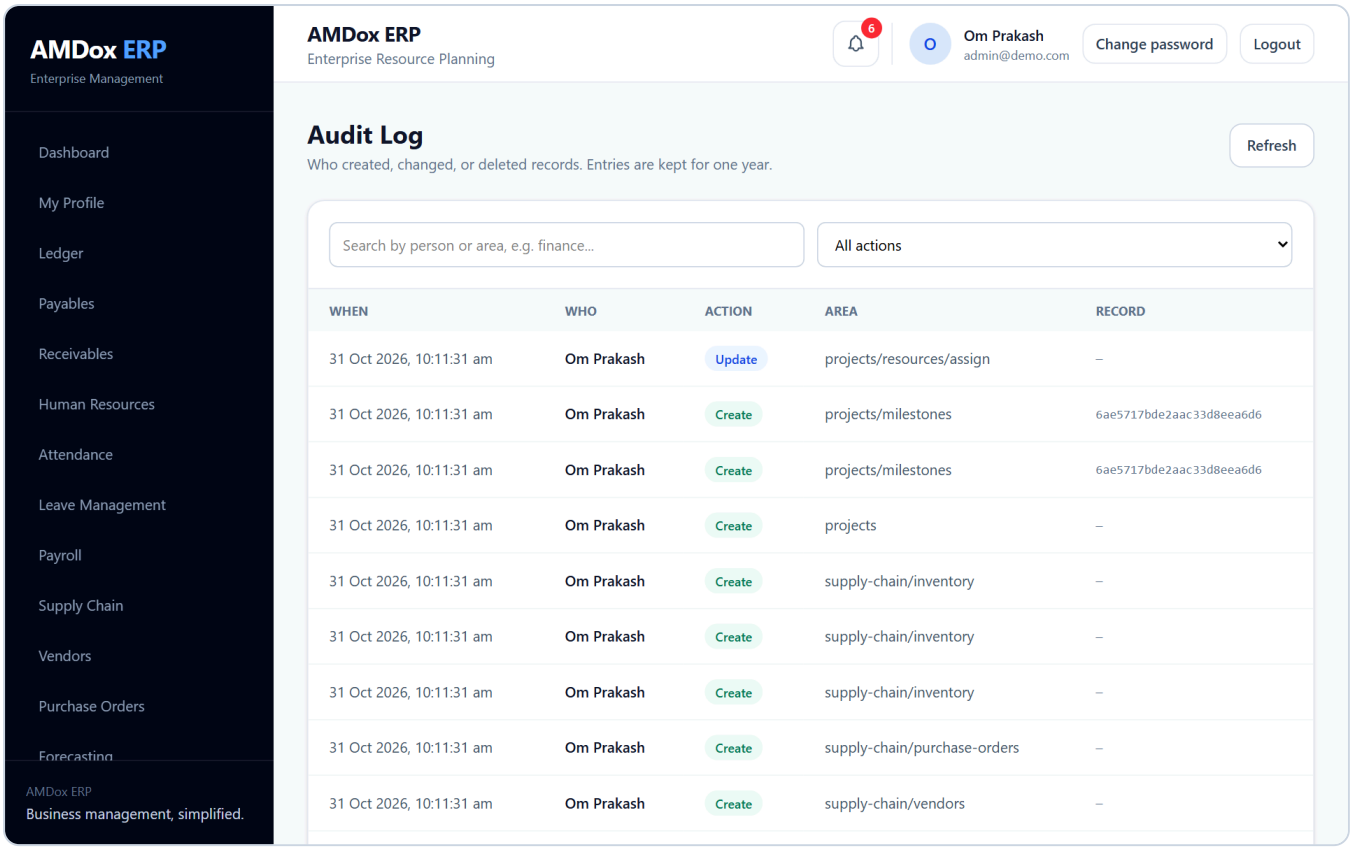
Projects — Project details, budget and milestones



Projects — Resource Planning

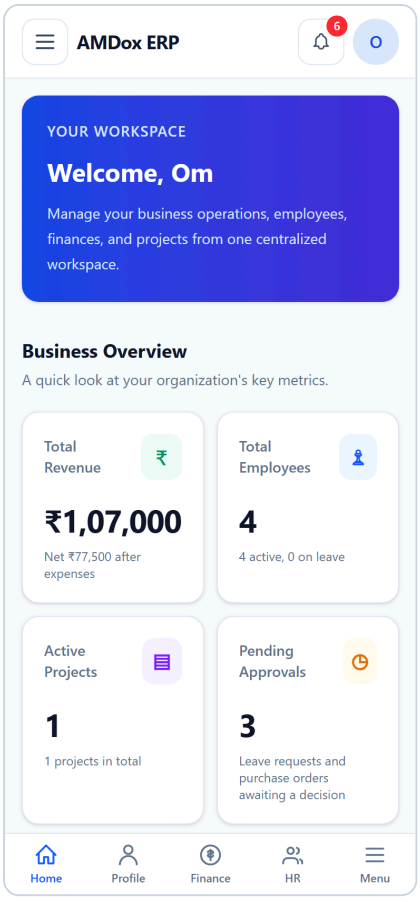


Settings — Roles and Permissions

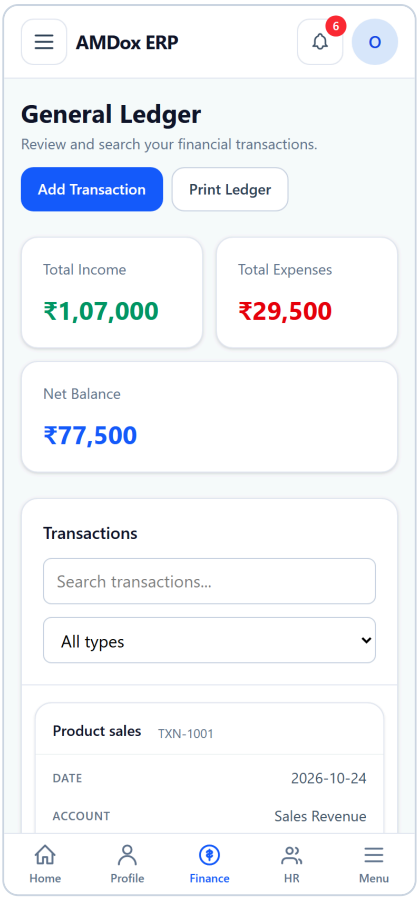


Settings — Audit Log

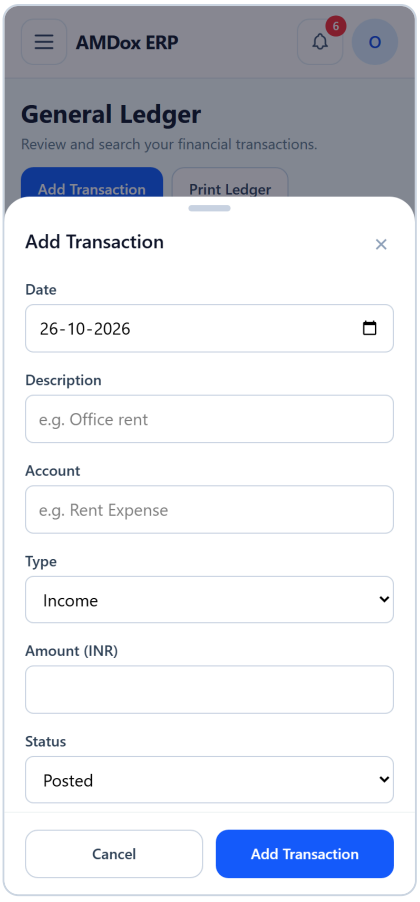
On a phone



Dashboard and tab bar



Ledger



Popup as a bottom sheet

7. Challenges and Learnings

Challenge	How it was solved
The original screens only showed sample data	Built a real backend and database, then connected every screen to it, one module at a time.
Keeping data private between roles	Permissions are checked on the server for every request, and tested automatically.
Fair approvals	Added rules so nobody can approve their own leave or a request they entered.
Hosting and connecting three cloud services	Configured environment variables, allowed origins (CORS), and the proxy setting for correct rate limiting.
Reloading a page on the live site showed "404"	Added a Vercel rewrite so every address is handled by the app.
Wide tables on small phones	Tables turn into cards on phones; popups became bottom sheets.

Key learnings

- Plan the data and the rules first; screens are easier once the API is clear.
- Security must live on the server, not only in the user interface.
- Automated tests make it safe to keep improving the project.
- Small branches and pull requests keep the work organised and easy to review.

8. Limits and Future Work

- The free server sleeps when idle, so the first visit can take about a minute.
- No emails are sent yet (for example, "forgot password").
- Integrations are listed but not connected to outside services yet.
- Forecasting uses a simple rule (demand plus expected growth), not machine learning.

Next steps: paid hosting for real use, email notifications, real integrations (payments, chat), and feedback from a trial with real users.

9. Conclusion

AMDox ERP started as a set of screens with sample data and is now a complete, working business application. All planned modules work with real data, permissions are enforced on the server, the app works well on phones, it is covered by 120 automated tests, and it is live on the internet. It is ready for a demonstration and for a trial with real users.

Live application: https://amdox-erp-psi.vercel.app
Source code: https://github.com/omjyotiprakash/amdox-erp